
Parking Lot Occupancy Detection System

A Complete Guide to Building a Real-Time Parking Spot Classifier
Using Computer Vision, Image Classification, and Smart Movement
Detection

Project Type	Computer Vision - Image Classification
ML Model	SVM (Support Vector Machine) with scikit-learn
Video Processing	OpenCV (cv2) with frame extraction
Key Technique	Binary mask + connected components for spot detection
Optimization	Movement-based classification (diff threshold)
Dataset	Parking_Lot_Detection_Counter (Kaggle)

TABLE OF CONTENTS

1. Project Overview

- 1.1 What This System Does
- 1.2 Why This Approach Works
- 1.3 Architecture Overview

2. Dataset and Project Structure

- 2.1 Parking_Lot_Detection_Counter Dataset
- 2.2 Folder Structure
- 2.3 The clf-data Directory

3. Understanding the Parking Spot Mask

- 3.1 What Is a Mask?
- 3.2 Why the Mask Is Essential
- 3.3 Creating a Mask (3 Methods)
- 3.4 Do You Need a New Mask for Each Parking Lot?

4. Training the Classifier

- 4.1 Training Workflow (6 Steps)
- 4.2 Training Code
- 4.3 Model Saving (Pickle)

5. Inference Pipeline (main.py)

- 5.1 Basic Pipeline: Classify Every Frame
- 5.2 Optimized Pipeline: Classify Every N Frames
- 5.3 Smart Pipeline: Movement-Based Classification

6. Movement Detection and Threshold Selection

- 6.1 calc_diff() Explained
- 6.2 Why Plotting Matters for Threshold Selection
- 6.3 Relative Threshold (Top 40%)

7. Utility Functions (util.py)

8. Reusing This System for a New Parking Lot

- 8.1 Step-by-Step Reuse Checklist
- 8.2 What Changes vs What Stays the Same

9. Key Lessons and Best Practices

1. PROJECT OVERVIEW

1.1 What This System Does

This system detects whether individual parking spots in a parking lot are **empty** or **occupied** using a camera feed (CCTV footage or video file). It processes video frames, extracts each parking spot region using a binary mask, and classifies each spot using a trained machine learning model.

The system displays real-time visual feedback with color-coded rectangles: **GREEN = Empty**, **RED = Occupied**, and a live counter showing available spots out of total spots.

1.2 Why This Approach Works

The key insight is that **video is just a sequence of image frames**. A model trained on still images of individual parking spots (cropped from overhead views) works perfectly on video by extracting the same regions from each frame. The **mask is the bridge** between training images and live video: the same spot coordinates used to crop training data are used to crop video frames at inference time.

1.3 Architecture Overview

Stage	Component	Description
1. Input	Binary Mask (.png)	Defines where parking spots are (white regions)
2. Input	Video File (.mp4)	CCTV footage or parking lot recording
3. Detection	connectedComponentsWithStats	Labels each white region as a separate spot
4. Extraction	get_parking_spots_bboxes()	Extracts bounding box (x, y, w, h) per spot
5. Movement	calc_diff()	Compares current frame vs previous frame per spot
6. Classification	empty_or_not()	SVM model predicts empty (True) or occupied (False)
7. Visualization	cv2.rectangle() + putText()	Color-coded rectangles + available count

2. DATASET AND PROJECT STRUCTURE

2.1 Parking_Lot_Detection_Counter Dataset

This project uses the **Parking_Lot_Detection_Counter** dataset from Kaggle, which is a perfect fit because it includes **both images AND video**, plus a pre-made binary mask and utility functions.

File	Purpose
mask_1920_1080.png	Binary mask of all parking spots (white = spot)
parking_1920_1080_loop.mp4	Full parking lot video for testing
mask_crop.png	Cropped mask (subset of spots, for quick testing)
parking_crop_loop.mp4	Cropped video (matches cropped mask)
clf-data/empty/	Training images of empty parking spots
clf-data/not_empty/	Training images of occupied parking spots
model.p	Pre-trained SVM model (pickle format)
util.py	Helper functions: empty_or_not(), get_parking_spots_bboxes()

2.2 Folder Structure

```
Parking System/
  parking/
    clf-data/
      empty/           (training images of empty spots)
      not_empty/       (training images of occupied spots)
      model/           (optional: saved model checkpoints)
      mask_1920_1080.png (binary mask for full video)
      mask_crop.png    (binary mask for cropped video)
      parking_1920_1080.mp4 (full parking lot video)
      parking_crop.mp4  (cropped parking lot video)
    venv/              (virtual environment)
    main.py            (inference pipeline)
    util.py            (helper functions)
    model.p            (trained model pickle)
    requirements.txt    (dependencies)
```

2.3 The clf-data Directory

The **clf-data** folder is the heart of the training pipeline. It contains cropped images of individual parking spots, organized into two classes. Each image is a small crop of one parking spot from overhead camera footage. The model learns to distinguish between these two classes during training.

Important: These are NOT full parking lot images. Each image shows only ONE parking spot, cropped using the mask coordinates. This is the same region that will be cropped from video frames during inference.

3. UNDERSTANDING THE PARKING SPOT MASK

3.1 What Is a Mask?

A mask is a **binary image** (black and white) that defines where the parking spots are in the camera frame. White pixels (255) represent parking spots, and black pixels (0) represent everything else (roads, sidewalks, buildings, trees).

The mask must be the **exact same resolution** as the video frames. For the full parking lot video (1920x1080), the mask is mask_1920_1080.png. For the cropped video, the mask is mask_crop.png. Using the wrong mask causes coordinate mismatches and crashes.

3.2 Why the Mask Is Essential

Without Mask	With Mask
Classify the ENTIRE frame	Classify ONLY parking spots
No way to know which spot is empty	Exact spot ID for each prediction
Wastes computation on roads, trees	Fast: classify fewer, smaller regions
One binary output for whole lot	Per-spot status with coordinates
Low accuracy	High accuracy per spot

3.3 Creating a Mask (3 Methods)

Method 1: Manual Drawing (Small lots, 5-20 spots)

```
import cv2
import numpy as np

img = cv2.imread('parking_lot_frame.jpg')
h, w = img.shape[:2]
mask = np.zeros((h, w), dtype=np.uint8)

# Draw white rectangles for each parking spot
cv2.rectangle(mask, (50, 100), (150, 200), 255, -1) # Spot 1
cv2.rectangle(mask, (160, 100), (260, 200), 255, -1) # Spot 2
cv2.imwrite('mask.png', mask)
```

Method 2: Interactive Mouse Drawing (Medium lots, 20-50 spots)

Use OpenCV's mouse callback to click and drag rectangles on a reference frame. This gives you precise control over spot placement. Use cv2.setMouseCallback() to register click events, then save the resulting coordinates as a mask image.

Method 3: Annotation Tools (Large lots, 50+ spots)

For large or complex parking lots, use dedicated annotation tools like **Roboflow Annotate** (web-based, free tier), **CVAT** (Computer Vision Annotation Tool), or **Labelling** (local tool). These

support polygon annotations for non-rectangular parking spots.

3.4 Do You Need a New Mask for Each Parking Lot?

YES, ALWAYS. Each parking lot has a unique layout, camera angle, spot dimensions, and geometry. The mask is **camera-position dependent**. However, you do NOT need a new mask for the same parking lot at different times of day, different weather, or different cars parked. The mask defines physical regions, not visual conditions.

Scenario	New Mask Needed?
Same lot, different time of day	NO (reuse)
Same lot, different weather	NO (reuse)
Same lot, different cars parked	NO (reuse)
Same lot, camera moved/adjusted	YES (new mask)
Different parking lot entirely	YES (new mask)
Same lot, different resolution video	YES (new mask)

4. TRAINING THE CLASSIFIER

4.1 Training Workflow (6 Steps)

The training workflow is the same for **every parking system project**. Only the data changes; the pipeline stays identical. You extract frames, crop spots, label them, train, validate, and save.

Step	Action	Details
1	Data Collection	Get video(s) of the parking lot OR use existing images from a dataset
2	Extract and Label	Take frames from video, crop spots using mask, label as empty/not_empty
3	Split Data	80% training set, 20% validation set
4	Train Model	Load images, resize to 15x15, flatten, train SVM/RandomForest/CNN
5	Validate	Test on unseen validation set, print confusion matrix, target 85%+ accuracy
6	Save Model	Save as pickle file (model.p) for loading at inference time

Key insight: You do NOT classify every frame during training. You take a few frames from the video, crop each parking spot, label them manually, and train. The training data is just individual spot images, not full frames.

4.2 Training Code

```
from sklearn.svm import SVC
import pickle, os, cv2
import numpy as np
from skimage.transform import resize

X_train, y_train = [], []

# Load empty parking spot images
for img_file in os.listdir('clf-data/empty/'):
    img = cv2.imread(f'clf-data/empty/{img_file}')
    if img is not None:
        img_resized = resize(img, (15, 15, 3))
        X_train.append(img_resized.flatten())
        y_train.append(0) # 0 = empty

# Load occupied parking spot images
for img_file in os.listdir('clf-data/not_empty/'):
    img = cv2.imread(f'clf-data/not_empty/{img_file}')
    if img is not None:
        img_resized = resize(img, (15, 15, 3))
        X_train.append(img_resized.flatten())
        y_train.append(1) # 1 = occupied

X_train = np.array(X_train)
y_train = np.array(y_train)

# Train SVM
model = SVC(kernel='rbf', C=1.0)
```

```
model.fit(X_train, y_train)

# Save
pickle.dump(model, open('model.p', 'wb'))
print('Model trained and saved!')
```

4.3 Model Saving (Pickle)

The trained model is saved as a **.p (pickle)** file. This serializes the entire trained SVM object, including learned weights and support vectors. At inference time, you simply load it with `pickle.load(open('model.p', 'rb'))`.

Version compatibility: Pickle files are tied to the scikit-learn version used during training. If you train with scikit-learn 1.1.3 and load with 1.9.1, you will get a version mismatch warning. Either downgrade scikit-learn to match, retrain with the current version, or suppress the warning (results are usually still valid for SVM models).

5. INFERENCE PIPELINE (main.py)

5.1 Basic Pipeline: Classify Every Frame

The simplest approach classifies every parking spot in every frame. This works but is **extremely wasteful**: for a 30 FPS video with 64 spots, that is 1,920 classifications per second. Most spots do not change between consecutive frames.

```
while ret:
    ret, frame = cap.read()
    for spot in spots:
        x1, y1, w, h = spot
        spot_crop = frame[y1:y1+h, x1:x1+w, :]
        spot_status = empty_or_not(spot_crop)
        if spot_status: # GREEN = empty
            cv2.rectangle(frame, ..., (0,255,0), 2)
        else:          # RED = occupied
            cv2.rectangle(frame, ..., (0,0,255), 2)
```

Important real-world note: This is NOT how it works in real-time. It takes 2-5 seconds to park a car, so classifying every single frame is unnecessary. You should classify every 1-3 seconds (30-90 frames at 30 FPS). This is a critical optimization for any parking system.

5.2 Optimized Pipeline: Classify Every N Frames

The first optimization is to classify only every N frames using a **step** variable. The classification results are cached in a `spots_status` list and reused for all intermediate frames.

Step Value	Time Interval	Pros	Cons
step = 30	~1 second	Fast updates	May still flicker
step = 60	~2 seconds	Less flicker	Slight delay
step = 90	~3 seconds	Best balance	3-sec lag on changes
step = 150	~5 seconds	Smoothest	Too slow for real-time

Recommended: **step = 60-90** (2-3 seconds). This captures all parking events while minimizing CPU usage and flickering.

5.3 Smart Pipeline: Movement-Based Classification

The smartest optimization is to **only classify spots where movement is detected**. In a real parking lot, most spots are in a steady state for hours (people shop or work). It is a waste of resources to keep classifying those static spots. Instead, we compare each spot region between the current frame and the previous frame, and only classify spots where the difference exceeds a threshold.

```
# Calculate movement per spot
if frame_nmr % step == 0 and previous_frame is not None:
    for spot_index, spot in enumerate(spots):
```

```
        x1, y1, w, h = spot
        spot_crop = frame[y1:y1+h, x1:x1+w, :]
        diffs[spot_index] = calc_diff(
            spot_crop,
            previous_frame[y1:y1+h, x1:x1+w, :]
        )

# Only classify high-movement spots
if previous_frame is None:
    arr_ = range(len(spots)) # First frame: classify all
else:
    arr_ = [j for j in range(len(spots))
            if diffs[j] / np.amax(diffs) > 0.4]

for spot_index in arr_:
    x1, y1, w, h = spots[spot_index]
    spot_crop = frame[y1:y1+h, x1:x1+w, :]
    status = empty_or_not(spot_crop)
    spots_status[spot_index] = status
```

6. MOVEMENT DETECTION AND THRESHOLD SELECTION

6.1 calc_diff() Explained

The `calc_diff()` function measures how much a parking spot changed between two frames by comparing the **average pixel intensity** of the same region across frames.

```
def calc_diff(im1, im2):
    return np.abs(np.mean(im1) - np.mean(im2))

# What it calculates:
# im1 = spot image from Frame N (all RGB pixels)
# im2 = spot image from Frame N+90 (same region)
# Returns: |avg_brightness(im1) - avg_brightness(im2)|

# Example:
# Empty spot brightness: avg = 140 (gray asphalt)
# Car parked brightness: avg = 80 (dark car body)
# Diff = |140 - 80| = 60 (BIG CHANGE = movement!)
```

Diff Value	Meaning	Action
0 - 1	Nearly static (same car, same lighting)	SKIP classification
1 - 3	Minor variations (shadows, slight movement)	SKIP (usually noise)
3 - 5	Noticeable change (car moving, lighting shift)	May classify
5+	Significant movement (car entering/leaving)	CLASSIFY this spot

6.2 Why Plotting Matters for Threshold Selection

Plotting is crucial to decide which threshold to pick. Instead of trial-and-error, you plot a histogram of all diff values collected from the video. The histogram shows two groups: static spots (left peak, low values) and moving spots (right tail, high values). The **gap between the two groups** is your optimal threshold.

```
import matplotlib.pyplot as plt

# After collecting all diffs from the video:
plt.hist(all_diffs, bins=50, edgecolor='black')
plt.xlabel('Diff Value')
plt.ylabel('Frequency')
plt.title('Distribution of Movement Diffs')
plt.axvline(x=0.3, color='r', linestyle='--', label='Threshold')
plt.legend()
plt.show()
```

Never guess the threshold. Always plot the distribution first, then choose the value at the natural gap between static and moving clusters. This is data-driven decision making.

6.3 Relative Threshold (Top 40%)

Instead of a fixed threshold, the current implementation uses a **relative threshold**: it normalizes each spot's diff by the maximum diff across all spots, then classifies only spots where the ratio exceeds 0.4 (40% of the maximum movement).

```
# Example with 8 spots:
diffs = [0.2, 1.5, 3.2, 0.1, 4.5, 2.1, 0.3, 5.0]
max_diff = 5.0

# Normalize: diffs[j] / max_diff
Spot 0: 0.2/5.0 = 0.04 -> 0.04 > 0.4? NO (skip)
Spot 1: 1.5/5.0 = 0.30 -> 0.30 > 0.4? NO (skip)
Spot 2: 3.2/5.0 = 0.64 -> 0.64 > 0.4? YES (classify)
Spot 3: 0.1/5.0 = 0.02 -> 0.02 > 0.4? NO (skip)
Spot 4: 4.5/5.0 = 0.90 -> 0.90 > 0.4? YES (classify)
Spot 5: 2.1/5.0 = 0.42 -> 0.42 > 0.4? YES (classify)
Spot 6: 0.3/5.0 = 0.06 -> 0.06 > 0.4? NO (skip)
Spot 7: 5.0/5.0 = 1.00 -> 1.00 > 0.4? YES (classify)

# Result: arr_ = [2, 4, 5, 7]
# Only classify 4 spots out of 8 = 50% CPU saved!
```

7. UTILITY FUNCTIONS (util.py)

The util.py file contains two core functions that handle model loading and spot extraction:

```
import pickle
import warnings
from skimage.transform import resize
import numpy as np
import cv2

EMPTY = True
NOT_EMPTY = False

warnings.filterwarnings('ignore', category=UserWarning)
MODEL = pickle.load(open('model.p', 'rb'))

def empty_or_not(spot_bgr):
    """
    Classify a single parking spot image.
    Args: spot_bgr (np.ndarray): Cropped BGR image of one spot
    Returns: True if empty, False if occupied
    """
    flat_data = []
    img_resized = resize(spot_bgr, (15, 15, 3))
    flat_data.append(img_resized.flatten())
    flat_data = np.array(flat_data)
    y_output = MODEL.predict(flat_data)
    if y_output == 0:
        return EMPTY
    else:
        return NOT_EMPTY

def get_parking_spots_bboxes(connected_components):
    """
    Extract bounding boxes for each parking spot from mask.
    Args: connected_components: Output of connectedComponentsWithStats
    Returns: List of [x, y, width, height] for each spot
    """
    (totalLabels, label_ids, values, centroid) = connected_components
    slots = []
    for i in range(1, totalLabels): # Skip background (0)
        x1 = int(values[i, cv2.CC_STAT_LEFT])
        y1 = int(values[i, cv2.CC_STAT_TOP])
        w = int(values[i, cv2.CC_STAT_WIDTH])
        h = int(values[i, cv2.CC_STAT_HEIGHT])
        slots.append([x1, y1, w, h])
    return slots
```

8. REUSING THIS SYSTEM FOR A NEW PARKING LOT

This is a **very modular project**. When you want to use it on a different parking lot, you just need CCTV footage and follow the same steps. The pipeline and code stay the same. Only the data and mask change.

8.1 Step-by-Step Reuse Checklist

Step	Action	Time Estimate
1	Get CCTV footage or video of the new parking lot	Already available
2	Extract ONE reference frame from the video	1 minute
3	Create a NEW binary mask for this lot (draw spots)	15-30 minutes
4	Verify mask resolution matches video resolution	1 minute
5	Extract frames, crop spots using mask, label empty/not_empty	30-60 minutes
6	Train a new classifier on the labeled data	5-10 minutes
7	Save model as model.p	1 minute
8	Update mask_path and video_path in main.py	1 minute
9	Run main.py and verify results	5 minutes
10	Plot diffs histogram and adjust threshold if needed	10 minutes

8.2 What Changes vs What Stays the Same

Component	Changes?	Details
main.py	Minimal	Only update mask_path and video_path
util.py	NO	Same functions work for any parking lot
mask.png	YES (always)	New mask for each parking lot / camera
model.p	YES (retrain)	Train new model on new lot's data
clf-data/	YES (new data)	Crop spots from new lot's footage
Pipeline logic	NO	Same connected components, same loop
calc_diff()	NO	Same movement detection algorithm
Threshold	MAYBE	Plot histogram and adjust if needed

The modularity is the strength of this project: swap the data and mask, retrain the model, and everything else works out of the box. You do NOT need to rewrite any code.

9. KEY LESSONS AND BEST PRACTICES

1. Understand the domain before coding

Parking lots are mostly static. Cars park for hours. Classifying every frame is wasteful. Understanding how parking works led to the movement-based optimization.

2. The mask is the bridge between training and inference

The same coordinates crop training images and video frames. Without the mask, there is no way to know which individual spot is empty or occupied.

3. Match mask resolution to video resolution

Using `mask_crop.png` with the full 1920x1080 video causes coordinate mismatches and crashes. Always verify that mask and video dimensions match.

4. Do not classify every frame

It takes 2-5 seconds to park a car. Classify every 2-3 seconds (step=60-90 at 30 FPS) for the best balance of responsiveness and CPU efficiency.

5. Plot before picking a threshold

Never guess. Plot the histogram of diff values, find the natural gap between static and moving spots, and set the threshold at that gap. This is data-driven, not trial-and-error.

6. Use relative thresholds for robustness

A relative threshold ($\text{diffs}[j] / \text{max_diff} > 0.4$) adapts to different lighting conditions and parking lots automatically, unlike a fixed threshold.

7. Previous frame is None on the first iteration

Always check if `previous_frame` is not `None` before comparing frames. The first frame has nothing to compare against; classify all spots on the first pass.

8. `cv2.imshow()` takes a string as the first argument

A common mistake: `cv2.imshow(frame, frame)` crashes because the first argument must be a window name string, not an image array.

9. Pickle version compatibility

Models saved with one scikit-learn version may not load cleanly on another. Either match versions, retrain, or suppress the warning.

10. This project is highly modular

To deploy on a new parking lot: get CCTV footage, create a new mask, label 50-100 spot images, retrain the classifier, update file paths. Everything else stays the same.

Dependencies (requirements.txt)

```
opencv-python==4.8.0.76  
numpy==1.24.3  
scikit-image==0.21.0  
scikit-learn==1.3.0  
matplotlib==3.7.2
```

End of Guide